Magongoa L.M (221478770)

Role: Incident Management Module Developer

Introduction

This semester, Internet Programming (INT316D) has significantly expanded my knowledge of web-based application development. Before enrolling in this module, I had only worked with basic front-end technologies like HTML and CSS, along with some foundational Java programming. The module pushed me much further, introducing me to the world of enterprise-level web development using Java Enterprise Edition (JEE). Key areas we covered included the historical progression of JEE, the MVC design pattern, JavaServer Pages, Servlets, exception handling strategies, Enterprise JavaBeans, the Java Persistence API, JPQL, and how to secure web applications effectively.

As part of our practical group project, we developed SmartFlow, a real-time traffic management system designed for the Witbank area. The purpose of the system was to improve how traffic incidents are managed and monitored. My responsibility in the project was to design and implement the Incident Management Module. This module allows traffic officers to view incidents, update their status, and mark incidents as resolved. This reflection explains my contribution to the project and demonstrates how I applied the concepts learned in INT316D.

My Contribution to SmartFlow Project

The SmartFlow project was created to solve the problem of slow communication and poor incident tracking in the Witbank traffic system. In many situations, emergency services and traffic officers receive information too late because there is no centralized platform for reporting and monitoring incidents. Our system was designed to provide a digital solution where incidents could be managed efficiently.

My responsibility was the Incident Management Module. I developed a page that displays all traffic incidents in a structured table format. The table contains important information such as incident ID, incident type, severity level, location, status, date reported, and reporter details. I also implemented functionality that allows officers to update the status of incidents from “Active” to “Investigating,” “Resolved,” or “False Alarm.” In addition, officers can mark incidents as resolved using a quick Resolve button.

To improve usability, I added different interface controls including radio buttons for filtering incidents by severity and a date picker for selecting the resolution date. I also integrated a stored procedure that updates the incident status and records the resolution timestamp automatically in the database. This module supports traffic officers by helping them respond to incidents more efficiently and track progress in real time.

Application of INT316D Topics

Evolution of JEE

One of the first concepts I learned in INT316D was the evolution of Java Enterprise Edition technologies. Understanding how Java web technologies developed over time helped me understand why enterprise applications are structured the way they are today. Early servlets mixed Java code with HTML, which made applications difficult to maintain. JSP improved presentation handling, while MVC introduced proper separation between presentation, business logic, and data access.

While developing my incident module, I applied these concepts by separating responsibilities correctly. The servlet handles request processing, the JSP handles presentation, and the Java classes represent the data model. Learning the evolution of JEE helped me appreciate why layered architectures are important for scalability and maintainability.

Model-View-Controller (MVC)

The MVC architecture played a major role in my module. The Model is represented by the Incident class and the database tables storing incident information. The View is the incidents.jsp page, which displays the data using HTML and JSP tags. The Controller is the IncidentServlet, which processes requests, communicates with the database, and forwards information to the JSP page.

Using MVC improved the organization of my project because each component had a clear responsibility. When I needed to change the display of the incidents table, I only edited the JSP file without affecting the servlet or database code. This separation also made debugging much easier.

JSP and Servlets

The JSP and Servlet components were central to the development of my module. The incidents.jsp page displays incident records dynamically by using data sent from the servlet. The IncidentServlet processes both GET and POST requests. The GET request retrieves incidents from the database and forwards them to the JSP page, while the POST request handles updates made by officers.

I learned that servlets act as the controller between the user interface and the business logic. They process form data, validate user input, and communicate with the database. JSP pages are mainly responsible for presenting information to the user. Understanding the relationship between JSP and servlets helped me understand how enterprise web applications function internally.

Exception Handling

Exception handling was very important in my project because database operations can fail for many reasons. For example, database connections may fail, SQL syntax may contain errors, or invalid data may be submitted by users. To prevent the system from crashing, I implemented try-catch-finally blocks throughout my module.

If an error occurs, the system displays a user-friendly message instead of showing technical details or stack traces. I also ensured that database connections and resources are always closed properly in the finally block. In addition, I created validation checks to prevent officers from resolving incidents that were already marked as resolved. Proper exception handling improved the reliability and professionalism of the system.

Enterprise JavaBeans (EJB)

Although our project did not fully implement EJB due to time limitations, I gained valuable theoretical knowledge about Enterprise JavaBeans. I learned that EJB simplifies transaction management, security, and business logic in enterprise applications.

In a future version of SmartFlow, I would use a Session Bean to manage incident operations such as updating statuses and resolving incidents. EJB would automatically handle

transactions and roll back operations if errors occur. This would improve the scalability and reliability of the application.

Java Persistence API (JPA) and JPQL

Another important topic covered in INT316D was JPA and JPQL. I learned that JPA allows developers to map Java objects directly to database tables using annotations. Instead of writing large amounts of JDBC code, developers can use entity classes and JPQL queries.

Although our project mainly used JDBC for simplicity, I now understand that JPA would make the code cleaner and easier to maintain. JPQL queries are object-oriented and database independent, making enterprise applications more portable.

Application Security

Security was an important requirement in SmartFlow because only authorized users should access certain modules. I implemented role-based access control to ensure that only traffic officers and administrators could access the incident management page.

I also used prepared statements to prevent SQL injection attacks. Session management was implemented so that users must log in before accessing secure pages. Through this topic, I learned the importance of protecting user data and preventing unauthorized access in enterprise systems.

Challenges and Lessons Learned

One major challenge I faced was integrating the stored procedure used to resolve incidents. Initially, I attempted to use multiple update statements directly in the servlet, but this created problems when one statement succeeded and another failed. I solved this problem by creating a stored procedure that handles all updates together as a single transaction.

Another challenge involved displaying meaningful information to users. Instead of showing the reporter's numeric ID, I joined the users table to display the reporter's full name. This improved the usability of the interface.

Throughout the semester, I learned not only technical skills but also teamwork, communication, and problem-solving. Working in a group taught me the importance of collaboration and time management when building enterprise systems.

Conclusion

The SmartFlow project gave me practical experience in applying the concepts learned in INT316D. My Incident Management Module demonstrates the use of MVC architecture, JSP, servlets, exception handling, database integration, stored procedures, and application security. The project also improved my understanding of enterprise Java development and strengthened my confidence as a programmer. I am proud of my contribution to the project and grateful for the skills and experience I gained during this semester.